

Robótica

Informe capítulo 3 – RVC v2 Peter Corke

Tiempo y movimiento

Docente: Jaime Enrique Arango Castro

Estudiante:

Cristhian Mateo Almeida Gómez

Universidad Nacional de Colombia

Sede Manizales



Resumen

Este capítulo introduce conceptos fundamentales sobre cómo representar, analizar y generar movimiento en robótica a lo largo del tiempo

3.1 Pose variable en el tiempo.

- **3.1.3 Transformación de velocidades espaciales**

Una velocidad especial es un vector de 6 elementos, los primeros 3 son velocidades lineales (traslación) y los últimos 3 son velocidades angulares (rotación).

En la primera parte se considera el caso en que un observador en un marco fijo llamado fA desea conocer la velocidad espacial de un objeto cuya velocidad está medida desde un segundo marco fB. La transformación se realiza usando una matriz conocida como jacobiano o matriz de interacción, que depende exclusivamente de la orientación entre los marcos, pero no de la traslación entre ellos. En términos prácticos, si la transformación entre los frames se describe con un objeto SE3, como en el siguiente código:

```
3] aTb = SE3.Tx(-2) * SE3.Rz(-pi/2) * SE3.Rx(pi/2);
   ✓ 0.0s

4] bV = [1, 2, 3, 4, 5, 6];
   ✓ 0.0s

5] aJb = aTb.jacob();
   aJb.shape
   aV = aJb @ bV
   ✓ 0.0s

array([-3, -1,  2, -6, -4,  5])
```

Figura 1. Velocidades especiales con marcos de referencias diferentes.

El jacobiano se obtiene con el método `.jacob()` después al multiplicar este jacobiano por el vector de velocidad espacial medido en el frame fB se obtiene la velocidad equivalente desde el punto de vista del frame fA. Este resultado refleja cómo las componentes han sido rotadas y algunas de ellas invertidas de signo debido a la orientación relativa entre los ejes de ambos marcos. Por ejemplo, la velocidad lineal en el eje x del frame fB ha sido transformada a una velocidad en el eje y negativo del frame fA, debido a la rotación de 90 grados entre sus ejes.

Posteriormente, el libro analiza una situación diferente, donde los marcos fA y fB están unidos rígidamente y se mueven juntos. En este caso, ambos marcos pueden tener observadores que midan la velocidad del sistema desde su propia perspectiva. Para transformar las velocidades en este contexto, se utiliza una matriz llamada adjunta (obtenida con el método `.Ad()`), que sí depende tanto de la rotación como de la traslación entre los frames. Esta matriz captura no solo la reorientación de los vectores, sino también cómo las velocidades angulares pueden inducir

componentes de traslación en otros marcos debido a la distancia entre los orígenes, por ejemplo, si en el frame fB el cuerpo presenta una traslación pura:

```

aV = aTb.Ad() @ [1, 2, 3, 0, 0, 0]
[6] ✓ 0.0s
array([-3, -1, 2, 0, 0, 0])

```

Figura 2. Traslación pura.

lo que indica que solo ha cambiado la dirección de las velocidades lineales por efecto de la rotación, y no aparece componente angular porque en el frame original no existía. Si en cambio el cuerpo tiene una rotación pura en torno al eje x:

```

aV = aTb.Ad() @ [0, 0, 0, 1, 0, 0]
[7] ✓ 0.0s
array([0, 0, 2, 0, -1, 0])

```

Figura 3. Rotación pura.

Esto indica que la rotación ha producido una traslación aparente en el eje z del frame fA, debido a que su origen está desplazado del eje de rotación y, por tanto, describe un movimiento circular alrededor del eje x de fB. Finalmente, si se combina una traslación y una rotación en el frame fB, como en el vector:

```

aV = aTb.Ad() @ [1, 2, 3, 1, 0, 0]
[8] ✓ 0.0s
array([-3, -1, 4, 0, -1, 0])

```

Figura 4. traslación y una rotación en el frame fB.

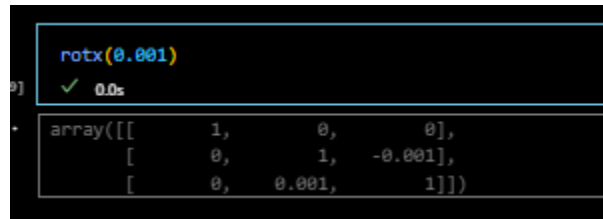
lo cual refleja tanto el reordenamiento de componentes como la interacción entre rotación y posición relativa de los frames. Este análisis muestra cómo, dependiendo del contexto y de la relación entre los marcos, deben utilizarse distintas herramientas matemáticas para transformar correctamente las velocidades espaciales. Mientras el jacobiano se aplica cuando se cambia el punto de vista sin movimiento relativo, la matriz adjunta se usa cuando los marcos se mueven juntos, pero desde posiciones diferentes. Ambos enfoques son esenciales en la planificación y control de movimiento en sistemas robóticos.

• 3.1.4 Rotación incremental

Cuando un objeto experimenta una pequeña rotación angular θ , esta puede expresarse como un vector tridimensional que representa tanto el eje de rotación como su magnitud. A diferencia de grandes rotaciones que requieren matrices o cuaterniones para representarse con precisión, las

rotaciones pequeñas pueden aproximarse utilizando una forma lineal, lo que simplifica los cálculos.

Esta sección del capítulo 3 continúa explorando las rotaciones incrementales, enfocándose en cómo se pueden aproximar y actualizar en el tiempo de forma eficiente. Para pequeñas rotaciones angulares, se puede usar una forma simplificada de la fórmula de Rodrigues, que aproxima una rotación como la suma de la matriz identidad y una matriz skew-symmetric construida a partir del vector de rotación. Por ejemplo, si se aplica una rotación muy pequeña alrededor del eje x:



```

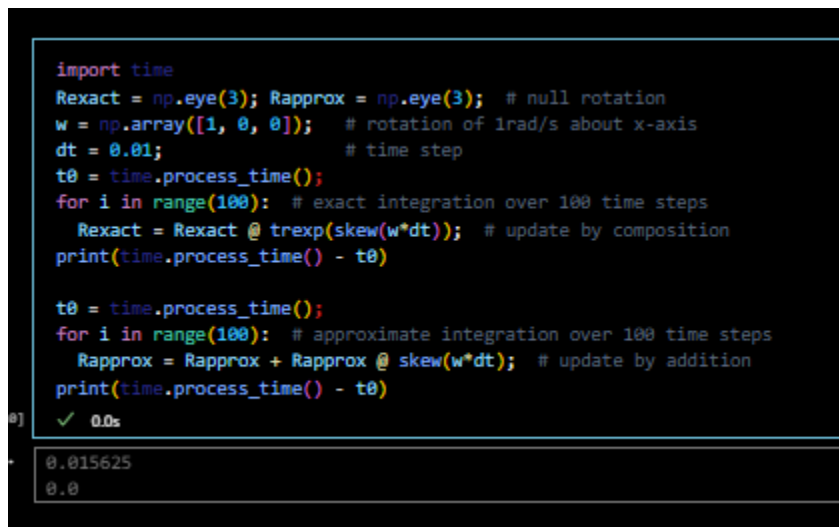
rotx(0.001)
✓ 0.0s
array([[ 1.,  0.,  0.],
       [ 0.,  1., -0.001],
       [ 0.,  0.001,  1.]])

```

Figura 5. Matriz de rotación para rotación incremental pequeña.

Esta matriz de rotación muestra que para un ángulo pequeño (0.001 radianes), los cambios en la matriz son muy pequeños y están localizados fuera de la diagonal principal. Esto refleja la idea de que una pequeña rotación puede ser representada como una identidad más una matriz skew-symmetric.

Esta aproximación es computacionalmente eficiente porque evita operaciones trigonométricas. Para comparar la eficiencia y precisión entre la integración exacta (usando la exponencial de matriz) y la integración aproximada (sumando incrementos), se realiza el siguiente experimento:



```

import time
Rexact = np.eye(3); Rapprox = np.eye(3); # null rotation
w = np.array([1, 0, 0]); # rotation of 1rad/s about x-axis
dt = 0.01; # time step
t0 = time.process_time();
for i in range(100): # exact integration over 100 time steps
    Rexact = Rexact @ trexp(skew(w*dt)); # update by composition
print(time.process_time() - t0)

t0 = time.process_time();
for i in range(100): # approximate integration over 100 time steps
    Rapprox = Rapprox + Rapprox @ skew(w*dt); # update by addition
print(time.process_time() - t0)
✓ 0.0s
0.015625
0.0

```

Figura 6. Experimento de eficiencia.

Primero, se realiza la integración exacta usando multiplicación de matrices de rotación. Luego, se repite el proceso, pero usando una integración aproximada por suma directa, este resultado muestra que la integración aproximada es alrededor de diez veces más rápida que la exacta. Sin embargo, al no pertenecer a un grupo de rotación ($SO(3)$), esta aproximación acumulativa puede llevar a una matriz de rotación inválida. Esto se puede verificar examinando su determinante:

```

[11] np.linalg.det(Rapprox) - 1
✓ 0.0s
... 0.010049662092876055

[12] np.linalg.det(Rexact) - 1
✓ 0.0s
... -2.886579864025407e-15

```

Figura 7. Comprobación de matriz de rotación válida.

El valor se aleja significativamente de 1, lo cual indica que Rapprox ya no es una matriz de rotación válida. En cambio, el resultado exacto muestra un error mucho menor, atribuible a errores de redondeo en la precisión numérica. No obstante, ambas matrices pueden normalizarse para volver a representar una rotación válida:

```

[13] tr2angvec(trnorm(Rexact))
✓ 0.0s
... (1.0, array([ 1, 0, 0]))

[14] tr2angvec(trnorm(Rapprox))
✓ 0.0s
... (0.999966668666524, array([ 1, 0, 0]))

```

Figura 8. Normalización de las matrices.

Ambas matrices representan una rotación de 1 radián alrededor del eje x, como se esperaba. Esto indica que, pese a las diferencias internas, el resultado final sigue siendo válido dentro de la precisión estándar de impresión.

3.2 Cuerpos acelerados y marcos de referencia

Cuando un observador se encuentra en un marco de referencia no inercial (es decir, uno que se acelera o rota), las leyes del movimiento deben ajustarse para tener en cuenta fuerzas ficticias o aparentes.

En particular, se analiza cómo la aceleración espacial (que incluye tanto la aceleración lineal como la angular) cambia dependiendo del marco desde el que se observa el cuerpo. Se derivan expresiones para calcular la aceleración de un cuerpo desde un marco de referencia acelerado, incluyendo términos adicionales como la aceleración de Coriolis y la aceleración centrífuga, que surgen debido a la rotación del marco de referencia.

- 3.2.1 Dinámica de los cuerpos en movimiento

Para el movimiento translacional, se aplica directamente la segunda ley de Newton en un marco inercial $\{0\}$ en cambio, para el movimiento rotacional en tres dimensiones, se usan las ecuaciones

de Euler. Estas describen cómo la aceleración angular en el marco del cuerpo depende del momento de inercia J_B , la velocidad angular ω_B , y el torque aplicado τ_B . Es importante destacar que, incluso si no hay torque aplicado, la velocidad angular no necesariamente permanece constante, a diferencia de la velocidad lineal. Esto se debe a la naturaleza compleja del momento angular, que puede conservarse en magnitud, pero cambiar en dirección.

Para observar este comportamiento, se simula el movimiento rotacional de un objeto en el espacio. Se define una matriz de inercia con términos no nulos fuera de la diagonal, lo que induce una dinámica de "tambaleo":

3.2.1 Dynamics of Moving Bodies

```
J = np.array([[ 2, -1, 0],
              [-1,  4, 0],
              [ 0,  0, 3]]);
orientation = UnitQuaternion(); # identity quaternion
w = 0.2 * np.array([1, 2, 2]);
```

[15] ✓ 0.0s

```
dt = 0.05; # time step
def update():
    global orientation, w
    for t in np.arange(0, 10, dt):
        wd = -np.linalg.inv(J) @ (np.cross(w, J @ w)) # (3.12)
        w += wd * dt
        orientation *= UnitQuaternion.EulerVec(w * dt)
    yield orientation.R

# tranimate(update(), dim=1.5);
plotvol3(new=True)
HTML(tranimate(update(), movie=True))
```

[16] ✓ 5.9s

Figura 9. Código de un movimiento de ‘tambaleo’.

se definen las condiciones iniciales, una orientación identidad representada con un cuaternión unidad y una velocidad angular inicial, el código define un generador en Python que simula la dinámica rotacional usando integración rectangular (también llamada Euler explícito). En cada paso de tiempo, se calcula la aceleración angular mediante la ecuación (3.14), y se actualizan tanto la velocidad angular como la orientación

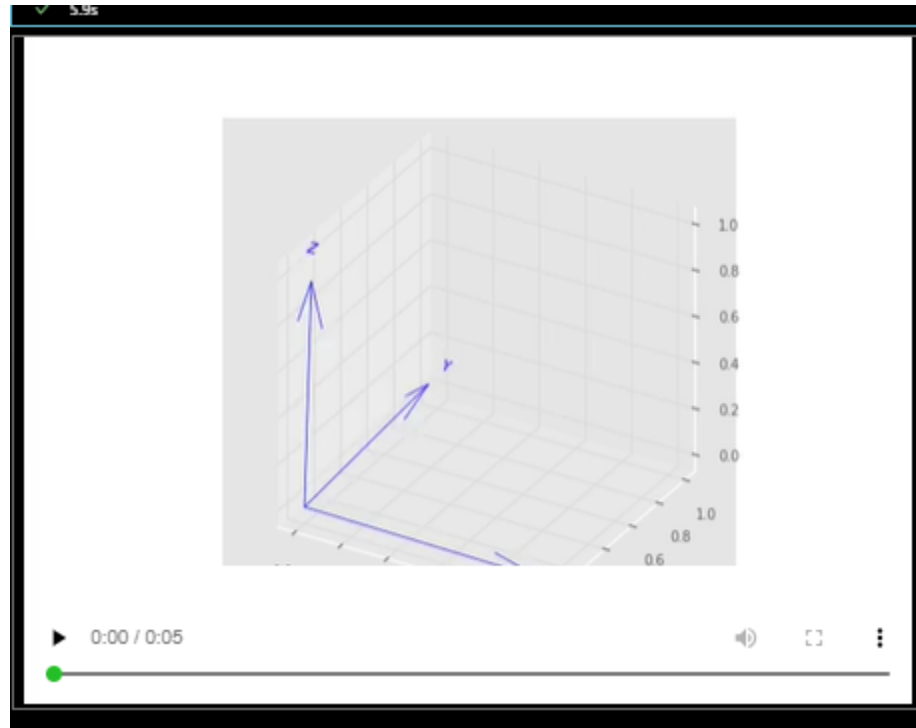


Figura 10. Posición inicial del FRAME.



Figura 10.1 Simulación de movimiento ‘tambaleo’.

Aquí, EulerVec genera un nuevo cuaternión a partir de un vector de Euler (una rotación pequeña), y tranimate anima la evolución de la orientación del cuerpo usando una representación gráfica del marco de coordenadas. Esta simulación muestra cómo un objeto puede rotar de forma compleja (incluso sin torque aplicado), debido a su distribución de masa y a los términos cruzados en su tensor de inercia. Esto ilustra cómo el comportamiento rotacional puede ser altamente no lineal, especialmente cuando el tensor de inercia tiene componentes fuera de la diagonal.

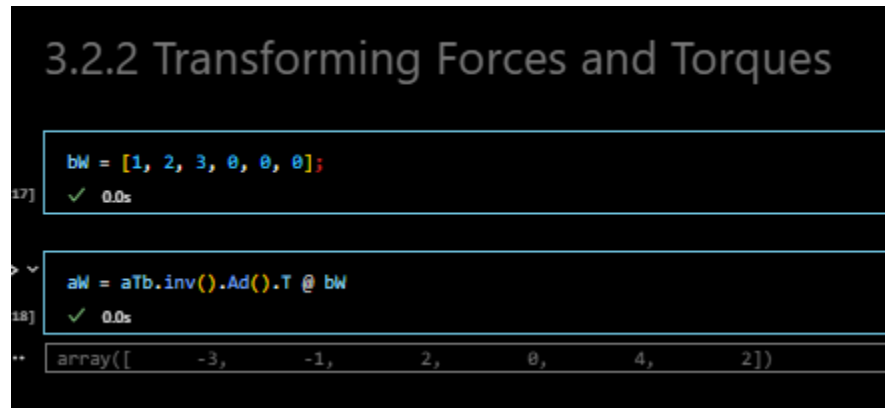
• 3.2.2 Transformación de fuerzas y pares

En esta sección se aborda cómo se representan y transforman las fuerzas y torques (también llamados momentos) aplicados a un cuerpo rígido dentro del marco de la mecánica espacial. Se introduce el concepto de wrench (torno o torca), que es un vector de 6 elementos que combina tanto la fuerza como el torque, Este vector representa completamente la acción mecánica que se

aplica a un cuerpo en un punto específico, normalmente en el origen de un sistema de coordenadas.

Un wrench definido respecto a un sistema de referencia $\{B\}$, y aplicado en su origen, se denota Bw . Si hay otro sistema de referencia $\{A\}$ fijo al mismo cuerpo, y se quiere conocer la wrench equivalente en ese sistema, se utiliza una transformación que asegura que ambos wrenches producirán el mismo efecto físico sobre el cuerpo, aunque estén expresados en marcos diferentes.

Esto se puede explorar con un ejemplo en código Python. Se define un wrench Bw que actúa con fuerzas puras (sin torque) a lo largo de cada uno de los ejes del sistema $\{B\}$:



```
3.2.2 Transforming Forces and Torques

In [17]: bw = [1, 2, 3, 0, 0, 0];
✓ 0.0s

In [18]: aw = aTb.inv().Ad().T @ bw
✓ 0.0s

Out[18]: array([-3, -1,  2,  0,  4,  2])
```

Figura 11. Transformación de fuerzas y pares.

Luego se transforma este wrench al sistema $\{A\}$, que está relacionado con $\{B\}$ por una transformación espacial aTb . Esto se hace aplicando el adjunto transpuesto de la transformación inversa, donde el resultado muestra que, aunque el wrench original solo tenía componentes de fuerza, la transformación produce componentes de torque en el sistema $\{A\}$. Esto se debe a que al cambiar el punto de aplicación de una fuerza (de un marco a otro), puede surgir un torque adicional debido al brazo de palanca generado por la distancia entre los orígenes de los marcos. En este caso, las fuerzas en $\{B\}$ causan un momento (torque) en $\{A\}$ de 4 Nm alrededor del eje y , y 2 Nm alrededor del eje z . No hay momento alrededor del eje x , lo que es consistente con la orientación relativa entre los marcos.

3.3 Creación de poses que varían en el tiempo

En robótica, es común necesitar que una pose (posición y orientación) varíe suavemente en el tiempo desde un estado inicial P_0 hasta uno final P_1 , como en el caso del movimiento deseado de un efector final o un dron. Esta suavidad implica que sus derivadas temporales, como velocidad y aceleración, sean continuas, y en algunos casos también la derivada de la aceleración (conocida como *jerk*). El problema se divide en dos partes: primero, construir una trayectoria suave en el espacio desde P_0 hasta P_1 , y segundo, definir un movimiento suave en el tiempo a lo largo de esa trayectoria.

- **3.3.1 Trayectorias unidimensionales suaves**

En esta sección se analiza cómo generar trayectorias suaves en una dimensión, es decir, una función escalar $q(t)$ que varía con el tiempo entre un valor inicial y uno final. Se busca que no solo la posición sea continua, sino también sus derivadas como velocidad y aceleración. Una forma común de lograrlo es usando un polinomio de quinto orden. Este tipo de polinomio tiene seis coeficientes, lo que permite satisfacer condiciones de frontera sobre posición, velocidad y

aceleración tanto al inicio como al final ($q_0, \dot{q}_0, \ddot{q}_0, q_t, \dot{q}_t, \ddot{q}_t$). Al aplicar estas ecuaciones en los instantes $t=0$ y $t=T$, se obtiene un sistema lineal de seis ecuaciones que permite resolver los coeficientes del polinomio. Esto solo es posible porque el número de incógnitas coincide con el de ecuaciones; por eso se elige un polinomio de quinto orden.

En Python, esto se implementa usando la función `quintic` del Toolbox. Por ejemplo:

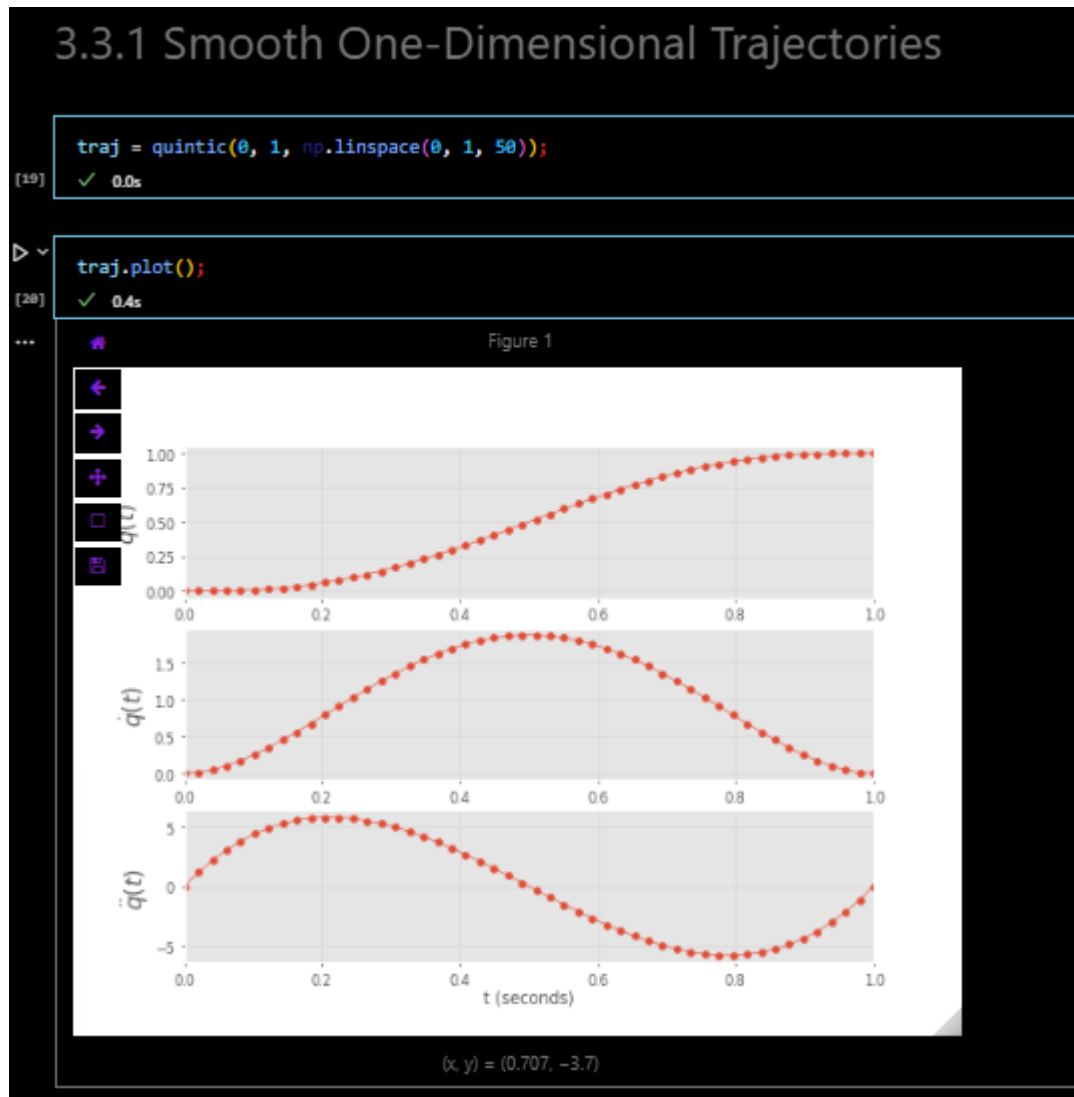


Figura 12. Posición, velocidad y aceleración continua.

Esto genera una trayectoria desde 0 hasta 1 en 50 pasos de tiempo. El objeto `traj` contiene tres atributos que representan la posición (`traj.q`), la velocidad (`traj.qd`) y la aceleración (`traj.qdd`), todos como arreglos de NumPy.

Si se desea imponer una velocidad inicial distinta de cero, como:

```

[21] quintic(0, 1, np.linspace(0, 1, 50), qd0=10, qdf=0);
✓ 0.0s

[22] qd = traj.qd;
      qd.mean() / qd.max()
✓ 0.0s

... 0.5231022222222196

```

Figura 13. Velocidad distinta de cero.

aunque la velocidad máxima puede ser alta, el promedio de velocidad puede ser bajo. En el ejemplo mostrado, la velocidad media es solo el 52% de la velocidad máxima esto implica que el motor no está siendo usado eficientemente durante toda la trayectoria.

Una alternativa muy utilizada es la trayectoria trapezoidal, que consta de tres fases: aceleración constante, velocidad constante, y desaceleración constante. Esto produce una forma trapezoidal en la gráfica de velocidad. Se implementa así:

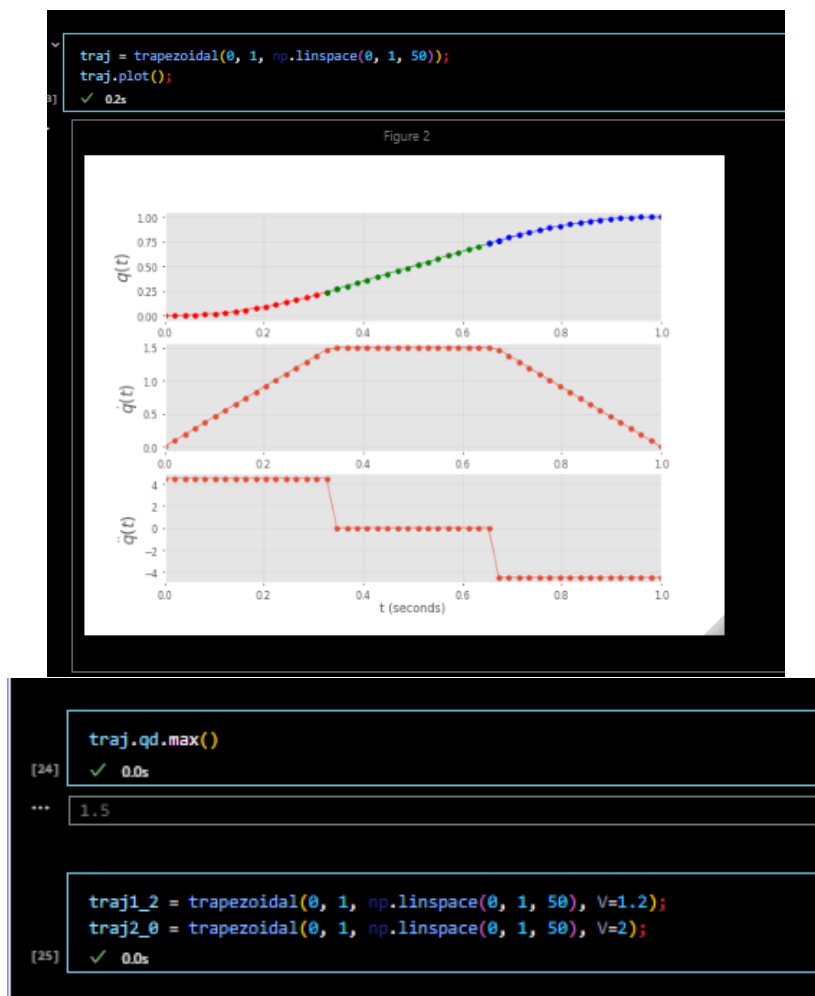


Figura 14. Trayectoria trapezoidal.

Esta función también permite especificar la velocidad máxima deseada, al aumentar la velocidad máxima, el segmento de velocidad constante se hace más corto. Si se especifica una velocidad demasiado alta o baja, la función puede generar un error por trayectoria inviable.

• 3.3.2 Trayectorias multieje

En la práctica, la mayoría de los robots tienen múltiples grados de libertad, lo que significa que su configuración no puede representarse con un solo valor escalar, sino con un vector $q \in \mathbb{R}^N$, donde N es el número de ejes de movimiento o grados de libertad. Por ejemplo, un robot con tres articulaciones tendrá una configuración $q=(q_0, q_1, q_2)$. En un robot móvil con ruedas, la configuración puede ser $q=(x,y)$ para posición o $q=(x,y,\theta)$ si se incluye la orientación. Para un cuerpo tridimensional con orientación en $SO(3)$, la configuración podría expresarse como ángulos de roll-pitch-yaw $q=(\alpha,\beta,\gamma)$ y para una pose en $SE(3)$, se podría usar $q=(x,y,z,\alpha,\beta,\gamma)$.

En todos estos casos, se requiere generar un movimiento suave en varias dimensiones, desde una configuración inicial hasta una final. Esta extensión del caso escalar se logra aplicando la misma técnica a cada dimensión por separado.

Por ejemplo, para un robot de dos ejes (tipo XY) que se mueve de la configuración (0,2) a (1,-1) en 50 pasos, se puede utilizar la función `mtraj` del Toolbox:



Figura 15. Múltiples trayectorias.

El resultado `traj.q` es un arreglo de NumPy de tamaño 50×250 , donde cada fila representa un paso de tiempo y cada columna corresponde a un eje del robot. El primer argumento (`trapezoidal`)

es una función escalar que genera trayectorias suaves en una dimensión, pero también se puede usar quintic. También se puede crear una trayectoria tridimensional al convertir un objeto SE3 a un vector de seis elementos combinando posición y orientación en ángulos de roll-pitch-yaw:

```
> T = SE3.Rand()
q = np.hstack([T.t, T.rpy()])
[28] ✓ 0.0s
array([ 0.09763,  0.4384,  0.2855,  2.777, -1.125,  1.781])
```

Figura 16. Trayectoria tridimensional.

• 3.3.3 Trayectorias multisegmento

En robótica, muchas aplicaciones requieren que un robot siga una trayectoria que pase por varios puntos intermedios o *waypoints* sin detenerse. Esto es común en tareas como evitar obstáculos, soldar una costura o aplicar sellador. Este problema está sobreconstrained, lo que significa que no es posible alcanzar exactamente cada punto intermedio si además se desea continuidad en la velocidad. Para lograr una transición suave entre segmentos, se sacrifican los pasos exactos por los puntos intermedios, y se utilizan trayectorias compuestas por segmentos lineales conectados mediante *blends* polinómicos suaves. En lugar del perfil trapezoidal, se utilizan polinomios de quinto orden (quintic) que permiten cumplir condiciones de frontera en posición, velocidad y aceleración.

El primer segmento de la trayectoria acelera desde q_0 con velocidad cero y se conecta con el segmento que se dirige a q_1 . La conexión entre segmentos ocurre mediante una mezcla polinómica iniciada a $t_{acc}/2$ antes de llegar a q_1 , con una duración de t_{acc} . Este proceso se repite para todos los segmentos. La velocidad constante $\dot{q}_0$ puede especificarse para cada tramo, y la aceleración media durante el *blend* se estima como:

$$\ddot{q} = \frac{\dot{q}_{k+1} - \dot{q}_k}{t_{acc}}$$

Si se conoce la aceleración máxima de un eje, se puede calcular el mínimo tiempo de *blend*. Aunque esta fórmula da una aceleración media, la aceleración pico real se determina usando la ecuación del polinomio quintic una vez se tienen sus coeficientes.

Como los ejes del robot pueden tener distintas velocidades máximas y diferentes distancias que recorrer en un segmento, primero se identifica qué eje tardará más en completar el segmento. Luego, se ajusta la velocidad de los demás ejes para asegurar que todos lleguen al siguiente punto al mismo tiempo, es decir, que el movimiento esté coordinado. Por ejemplo, se puede definir una trayectoria para un robot de dos ejes siguiendo los vértices de un cuadrado rotado:

```

via = S02(30, unit="deg") * np.array([[ -1,  1,  1, -1, -1], [ 1,  1, -1, -1,  1]]);
traj0 = mstraj(via.T, dt=0.2, tacc=0, qdmax=[2, 1]);
[29] ✓ 0.0s

xplot(traj0.q[:, 0], traj0.q[:, 1], color="red");
plt.plot(traj0.q[:, 0], traj0.q[:, 1], color="red");
[30] ✓ 0.2s

```

Figure 4

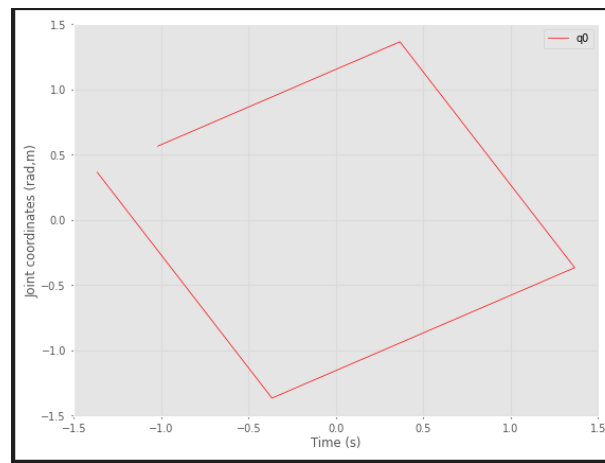
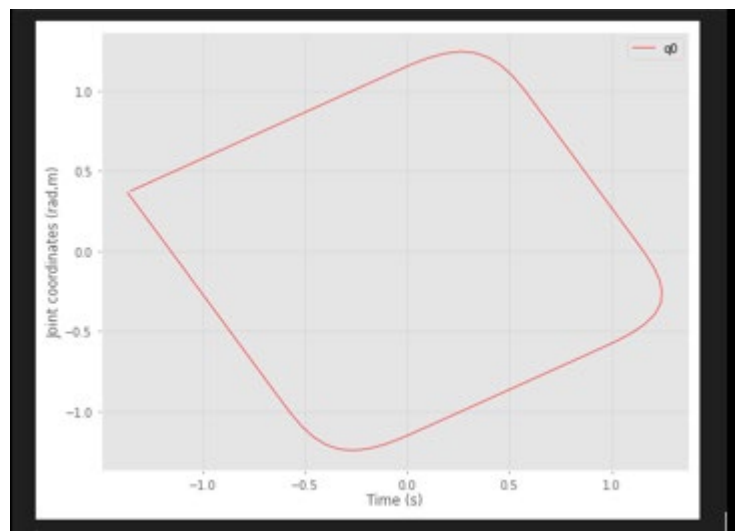


Figura 17. Cambios de trayectorias.

Aquí, `via.T` define los puntos intermedios como columnas, `dt` es el tiempo de muestreo, `tacc` es el tiempo de aceleración, y `qdmax` es la velocidad máxima por eje. La función devuelve un objeto con los datos de la trayectoria también se puede aumentar el tiempo de aceleración:



```
traj2 = mstraj(via.T, dt=0.2, tacc=2, qdmax=[2, 1]);
✓ 0.0s

len(traj0), len(traj2)
✓ 0.0s

(28, 80)
```

Figura 18. Aumento de aceleración.

la trayectoria es más suave, pero también toma más tiempo en completarse.

• 3.3.4 Interpolación de orientación en 3D

En robótica, frecuentemente es necesario interpolar entre dos orientaciones en 3D, por ejemplo, cuando se requiere que el efector final de un robot pase suavemente de una orientación R0 a otra R1 por ejemplo, se definen dos orientaciones:

```
R0 = S03.Rz(-1) * S03.Ry(-1);
R1 = S03.Rz(1) * S03.Ry(1);
✓ 0.0s

rpy0 = R0.rpy(); rpy1 = R1.rpy();
✓ 0.0s

traj = mtraj(quintic, rpy0, rpy1, 50);
✓ 0.0s

pose = S03.RPY(traj.q);
len(pose)
✓ 0.0s

50
```

Figura 19. Código de Interpolación de orientación en 3D.

Se obtienen sus ángulos roll-pitch-yaw equivalentes y se genera una trayectoria suave de 50 pasos usando interpolación quintica por ultimo con la función traj.q se hace un arreglo 50x3 con los ángulos interpolados. Para visualizar el movimiento, se convierten nuevamente a matrices de rotación. Cuando observamos la Figura 20 se aprecia que el movimiento es suave, puede parecer descoordinado si el cambio de orientación es grande, ya que el eje de rotación varía a lo largo de la trayectoria. También pueden surgir problemas si alguna de las orientaciones está cerca de una singularidad en esa representación.

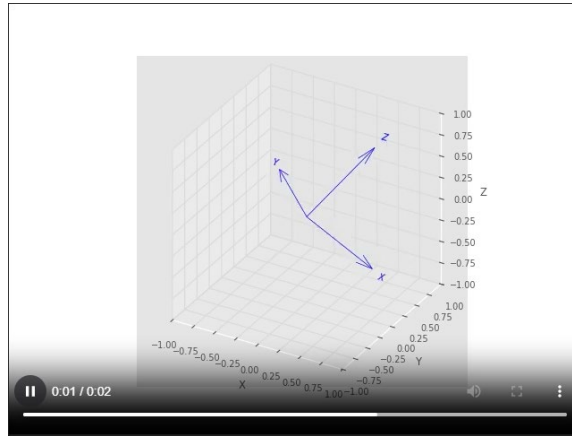


Figura 20. Animación de la interpolación de orientación.

La interpolación mediante cuaterniones unitarios es solo un poco más compleja y ofrece una rotación sobre un eje fijo en el espacio. Usando el Toolbox, primero se convierten las matrices de rotación a cuaterniones unitarios y se interpola en 50 pasos uniformes:

```

[38] q0 = UnitQuaternion(R0); q1 = UnitQuaternion(R1);
✓ 0.0s

[39] qtraj = q0.interp(q1, 50);
len(qtraj)
✓ 0.0s
50

> qtraj.animate()
plotvol3(new=True) # for matplotlib/widget
HTML(qtraj.animate(movie=True))
[40] ✓ 2.4s

c:\Users\MateoAlmeida\Desktop\10mo\robotic\localRep\roboticDev\lib\site-packages\matplotlib\animation.py:908
warnings.warn(

```

Figura 21. interpolación mediante cuaterniones.

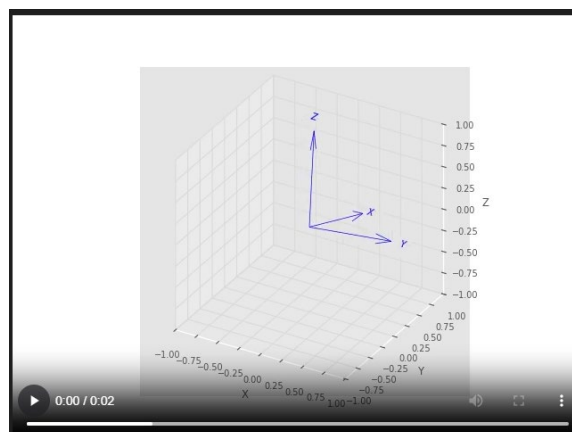


Figura 22. Animación de la interpolación mediante cuaterniones.

Esta interpolación se realiza usando *slerp* (spherical linear interpolation), donde los cuaterniones se mueven sobre un círculo máximo en una hipersfera de 4 dimensiones. En el espacio tridimensional, esto equivale a una rotación sobre un eje fijo.

• 3.3.4.1 Dirección de rotación

Cuando se rota entre dos puntos sobre un círculo, existen dos posibles trayectorias: una en el sentido horario y otra en el sentido antihorario. Aunque ambas alcanzan el mismo destino, la distancia angular recorrida puede ser distinta. Esta ambigüedad también se presenta al interpolar orientaciones en el espacio tridimensional usando cuaterniones unitarios, ya que estos representan rotaciones sobre la superficie de una hipersfera. Por ejemplo, si se desea interpolar una rotación alrededor del eje z , desde un ángulo de -2 radianes hasta $+2$ radianes, se puede hacer lo siguiente:

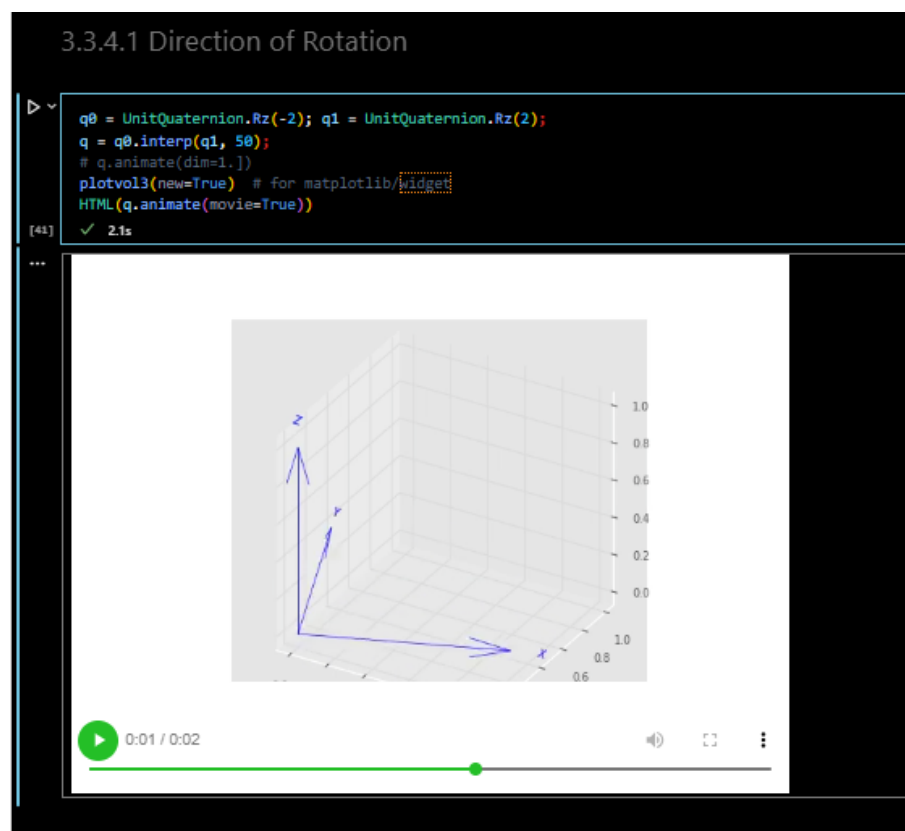


Figura 23. Código de dirección de rotación.

Este código genera una interpolación de 50 pasos que recorre la trayectoria larga sobre el círculo: una rotación de 4 radianes alrededor del eje z . Sin embargo, existe otra trayectoria más corta: en lugar de ir de -2 a $+2$ pasando por $\pm\pi$, se podría tomar el camino contrario recorriendo solo $2\pi - 4 \approx 2.282$ radianes. Para forzar la interpolación sobre la trayectoria más corta, se utiliza el argumento `shortest=True` como lo podemos observar en la figura 24:

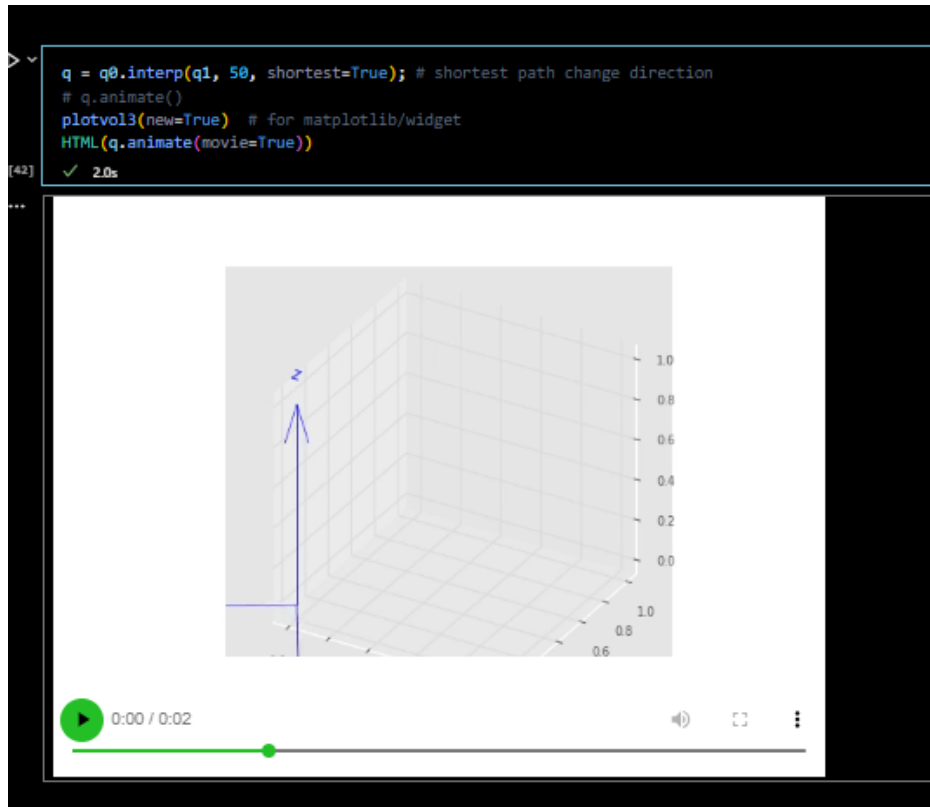


Figura 24. Reducción de trayectoria usando cuaterniones.

Esto produce una rotación que sigue el camino más directo en el espacio de las rotaciones. La diferencia es visible en la animación: en el primer caso, la orientación rota completamente en un sentido, mientras que, en el segundo rota en el sentido contrario, pero recorre una distancia angular menor.

• 3.3.5 Movimiento cartesiano en 3D

En esta sección se aborda un problema común en robótica: encontrar una trayectoria suave entre dos poses tridimensionales, es decir, que varíe tanto la posición como la orientación. Este tipo de movimiento se denomina movimiento cartesiano, ya que ocurre en el espacio $R^3 * S^3$, combinando translación y rotación. Las poses se representan mediante matrices homogéneas del grupo $SE(3)$, que contienen tanto la posición en el espacio como la orientación en forma de matriz de rotación.

Para ilustrar el proceso, se definen dos poses utilizando la clase $SE3$. La primera se ubica en la posición $[0.4, 0.2, 0]$ con una rotación de 3 radianes en yaw. La segunda está en $[-0.4, -0.2, 0.3]$ y con orientación especificada en ángulos roll-pitch-yaw como $-\pi/4$, $\pi/4$ y $-\pi/2$, respectivamente. Estas poses se combinan mediante $SE3.Trans()$ para la posición y $SE3.RPY()$ para la orientación:

```

43] T0 = SE3.Trans([0.4, 0.2, 0]) * SE3.RPY(0, 0, 3);
    T1 = SE3.Trans([-0.4, -0.2, 0.3]) * SE3.RPY(-pi/4, pi/4, -pi/2);
    ✓ 0.0s

44] T0.interp(T1, 0.5)
    ✓ 0.0s
**
-0.7205  0.6578  0.2196  0
-0.6578 -0.5479 -0.5168  0
-0.2196 -0.5168  0.8274  0.15
0 0 0 1

45] Ts = T0.interp(T1, 51);
    ✓ 0.0s

46] len(Ts)
    ✓ 0.0s
**
51

```

Figura 25. Movimiento cartesiano.

La interpolación entre ambas se realiza con el método `.interp()` al interpolar con $s=0.5$, se obtiene una pose intermedia ubicada en el punto medio del trayecto, tanto en posición como en orientación también esta interpolación combina una interpolación lineal de la parte de translación con una interpolación esférica de la rotación usando cuaterniones unitarios. Se puede generar una trayectoria completa entre las dos poses usando 51 pasos de interpolación, obteniendo una lista de transformaciones homogéneas que describen el movimiento paso a paso.

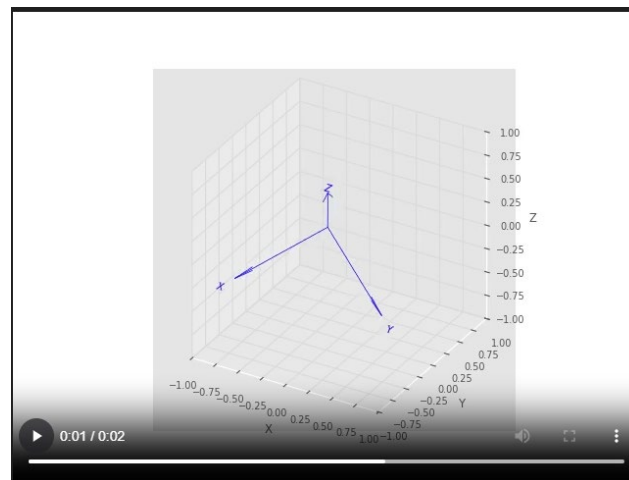


Figura 25. Movimiento cartesiano 3D.

Para analizar la evolución de la posición, se puede extraer la parte de translación con `.t`, que produce un arreglo de tamaño 51×3 .

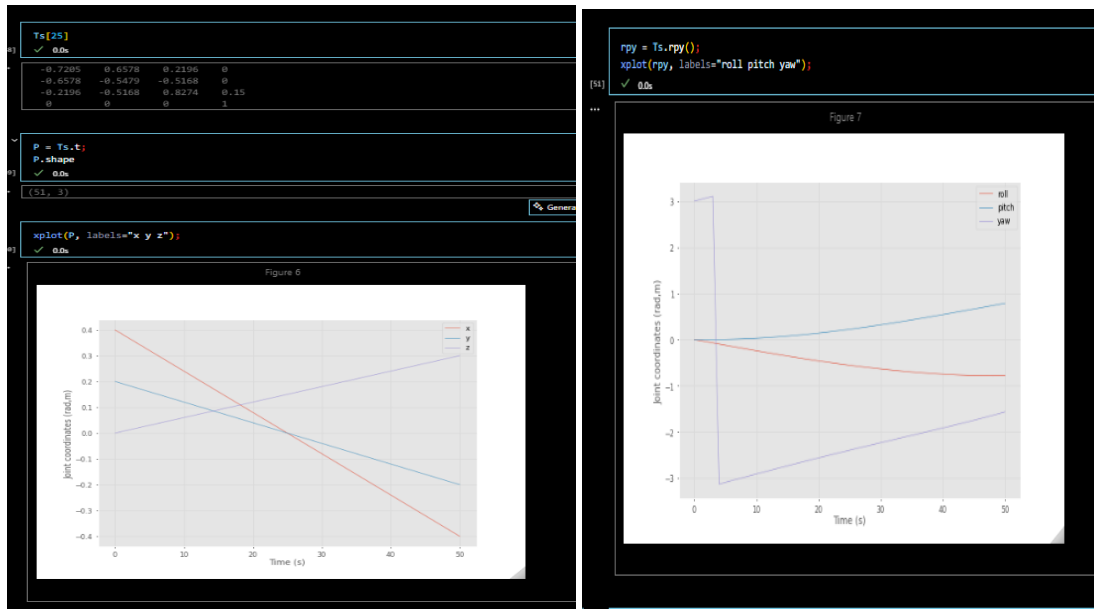


Figura 26. Coordenadas de posición.

Al graficarlo con `xplot()`, se observa que las coordenadas de posición varían linealmente con el tiempo. En cambio, cuando se grafican los ángulos roll-pitch-yaw obtenidos con `.rpy()`, se aprecia una variación no lineal. Esto se debe a que la conversión desde la orientación interpolada en cuaterniones a roll-pitch-yaw es una transformación no lineal. Además, en este ejemplo, la trayectoria pasa cerca de una singularidad en la representación de roll-pitch-yaw (alrededor del paso 24-25), lo cual se manifiesta en un cambio rápido y abrupto de los valores de estos ángulos, aunque en la animación no se observe un aumento real en la velocidad de rotación.

Un aspecto importante a considerar es que, aunque el camino espacial es suave, el movimiento no lo es en términos de velocidad y aceleración. En los extremos del trayecto (inicio y final), la velocidad sufre saltos discontinuos, ya que se pasa bruscamente de velocidad cero a un valor finito y viceversa, sin una fase de aceleración ni desaceleración. Esto se puede solucionar utilizando funciones de interpolación temporal como `trapezoidal()` o `quintic()`, que generan perfiles suaves de la variable de interpolación `sss`. Estas funciones se pueden pasar como segundo argumento a `.interp()` en forma de vector, logrando que la velocidad y aceleración a lo largo de la trayectoria también sean suaves:

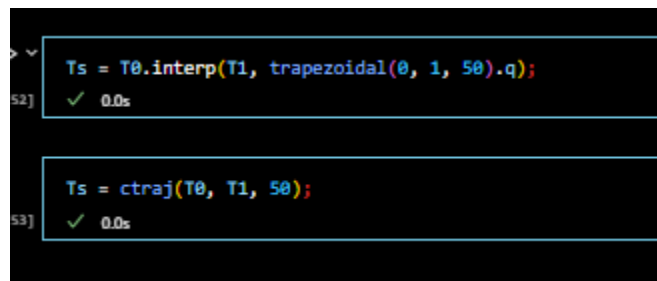


Figura 27. Suavización del movimiento.

El camino espacial permanece igual, pero ahora la velocidad a lo largo de la trayectoria es suave, con una fase de aceleración inicial y desaceleración final. Esto produce trayectorias más realistas y apropiadas para robots físicos.

3.4 Aplicación: Navegación inercial

Un sistema de navegación inercial (INS) es un dispositivo autónomo que estima su posición, velocidad y orientación midiendo aceleraciones y velocidades angulares, e integrándolas en el tiempo, sin necesidad de señales externas como GPS. Esto lo hace ideal para aplicaciones como submarinos, misiles o naves espaciales, donde no hay comunicación con ayudas de navegación por radio. Aunque los primeros INS eran grandes y costosos, hoy en día son pequeños, baratos y comunes, incluso en teléfonos inteligentes. El INS estima su movimiento respecto a un marco de referencia inercial, generalmente fijo a la Tierra, y utiliza un sistema de coordenadas adjunto al vehículo llamado marco del cuerpo (body frame).

- **3.4.1.2 Estimación de la orientación**

En la navegación inercial, una de las tareas principales es estimar la orientación del sistema a lo largo del tiempo. Esta orientación, también conocida como actitud, evoluciona a medida que el sensor se mueve y rota. Este proceso puede implementarse de forma más robusta y eficiente utilizando cuaterniones unitarios. Para simular este procedimiento, se utiliza el conjunto de datos proporcionado por el archivo `imu_data.py`, que contiene una secuencia simulada de velocidades angulares para un cuerpo en rotación.

```
[54] from imu_data import IMU
      true, _ = IMU()
      ✓ 0.2s

[55] orientation = UnitQuaternion(); # identity quaternion
      ✓ 0.0s

> for w in true.omega[:-1]:
    next = orientation[-1] @ UnitQuaternion.EulerVec(w * true.dt);
    orientation.append(next);
    len(orientation)
[56] ✓ 0.0s

*** 400
```

Figura 28. Estimación de la orientación.

La variable `true.omega` contiene una secuencia de vectores de velocidad angular en el marco del cuerpo, y `true.t` contiene los tiempos correspondientes. Se inicializa la orientación con el cuaternión identidad, lo que representa una orientación nula, luego se actualiza la orientación en cada paso de tiempo, multiplicando el cuaternión anterior por un nuevo cuaternión incremental, el cual se obtiene a partir del vector velocidad angular escalado por el intervalo de tiempo.

Cuando entramos al ciclo `for` la función `UnitQuaternion.EulerVec(w * dt)` devuelve un cuaternión que representa una rotación cuyo eje y ángulo están dados por el vector `w*dt`. El operador `@` realiza la multiplicación de cuaterniones, lo que equivale a componer rotaciones, y normaliza automáticamente el resultado. Después de ejecutar el bucle, la variable `orientation` contiene la historia completa de la orientación a lo largo de 400 pasos de tiempo.

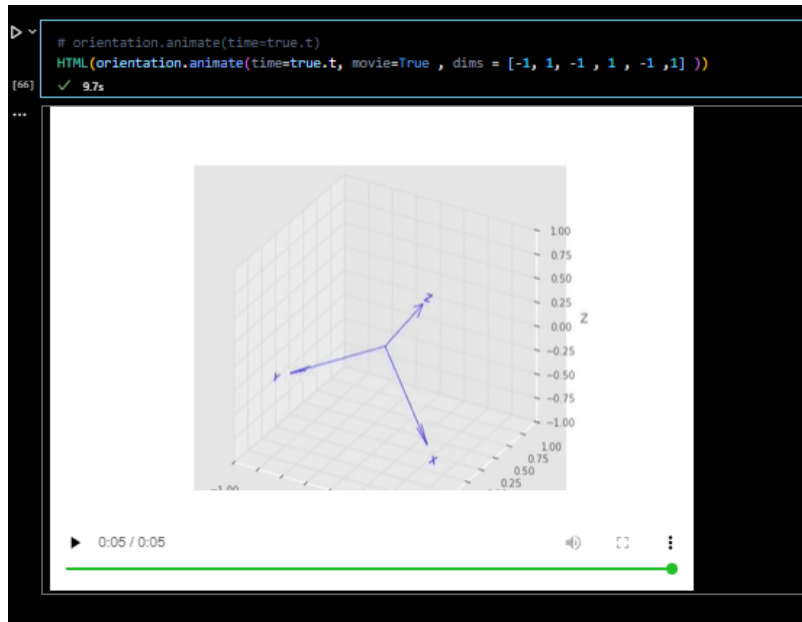


Figura 29. Animación de orientación.

En la Figura 30 se muestra cómo varían las componentes angulares en el tiempo. Aunque se puedan observar cambios rápidos o comportamientos no lineales, esto muchas veces es una consecuencia del sistema de representación (como los ángulos RPY), y no necesariamente de un aumento en la velocidad angular real del cuerpo.

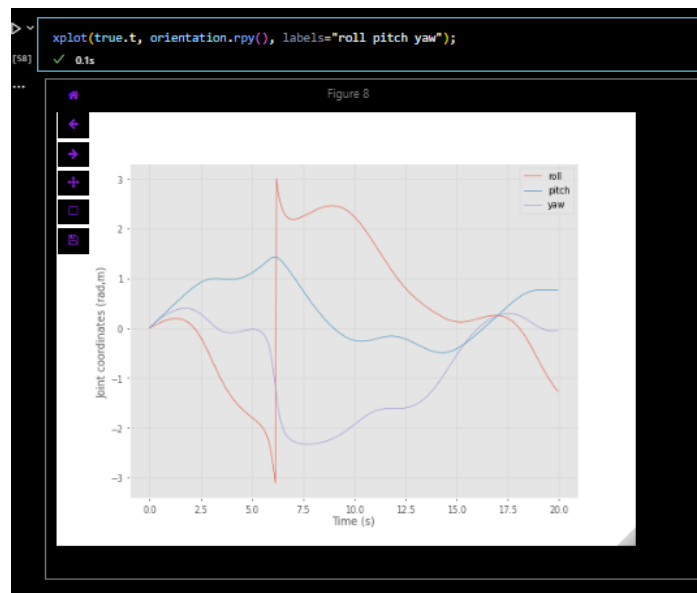


Figura 30. Variación angular en el tiempo.

• 3.4.4 Fusión de sensores inerciales

Los sistemas modernos, llamados strapdown, ya no utilizan gimbals. En lugar de eso, los sensores están directamente fijados al vehículo. Este conjunto de sensores se denomina unidad de medición inercial (IMU). Una IMU de 6 ejes contiene acelerómetros y giroscopios triaxiales; una IMU de

9 ejes incluye además un magnetómetro triaxial. Algunas versiones modernas también incorporan sensores de presión barométrica para estimar altitud.

La exactitud de medición de los sensores esta afectado por errores como un factor de escala, sesgo o bias(desviación sistemática) o ruido aleatorio. Los errores en la orientación también afectan la interpretación de la aceleración si el sistema cancela incorrectamente la gravedad, terminará integrando valores erróneos. El resultado es una aceleración falsa.

Esto muestra lo crítica que es la calibración del bias. Un bias de solo 0.01 grados por hora puede generar errores de posición de aproximadamente 1 milla náutica por hora (1.85 km/h). Mientras que sistemas militares tienen estabilidad de menos de 0.00002 grados por hora, los sensores de consumo pueden tener desviaciones de 0.01–0.2 grados por segundo. Para ilustrar el impacto del bias, se utiliza un ejemplo con datos simulados de un IMU:

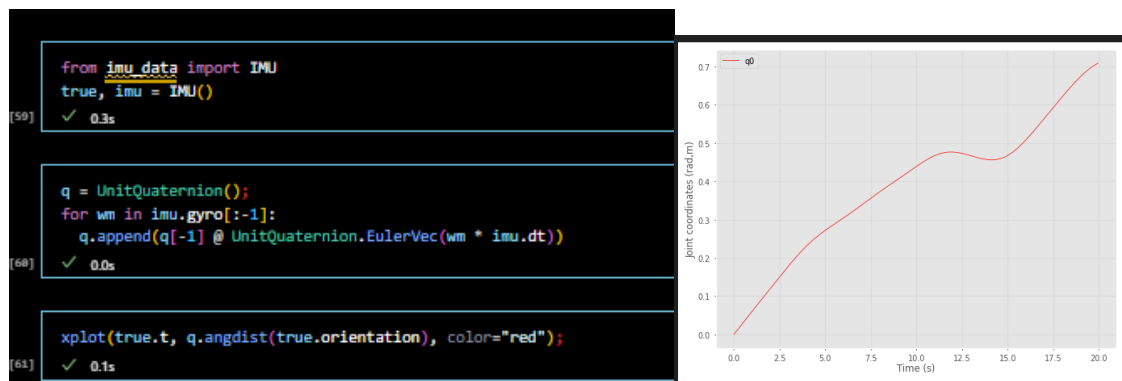


Figura 31. Fusión de sensores inerciales.

Se integra la orientación estimada usando los datos del giroscopio con bias incluido y se grafica el error angular entre la orientación verdadera y la estimada, el gráfico resultante muestra cómo el error en orientación crece con el tiempo debido al bias.

Una estrategia más sofisticada consiste en estimar el bias en línea (mientras el sistema está en funcionamiento), lo que se logra mediante fusión de sensores. Esta técnica combina acelerómetros, giroscopios y magnetómetros para aprovechar sus características complementarias. Una técnica común de fusión es el filtro de Kalman extendido, pero aquí se presenta una alternativa simple y eficaz: el filtro complementario explícito.

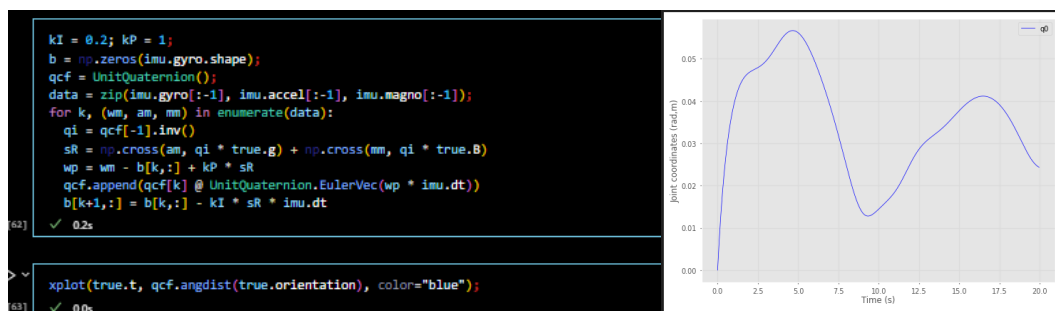


Figura 32. Disminución del error angular.

La curva azul en el gráfico muestra cómo el filtro complementario logra detener el crecimiento del error, incluso en presencia de bias. Además, las estimaciones del bias convergen hacia su valor real.

- Conclusiones

- El capítulo 3 consolida el vínculo entre la representación matemática del movimiento y su aplicación práctica mediante transformaciones, rotaciones incrementales y wrenches. Comprender la relación entre frames, jacobianos, adjuntas y fuerzas es esencial para diseñar sistemas de control robustos en robótica móvil y manipuladores.
- La correcta integración de velocidades, aceleraciones y fuerzas, sumada a la interpolación suave de trayectorias, garantiza un comportamiento físico realista y controlado.
- Estos fundamentos son la base de implementaciones prácticas en ROS2, donde se combinan sensores inerciales, cuaterniones y modelos dinámicos para navegación autónoma precisa, incluso en entornos sin GPS.

- Referencias

- Corke, P. Robotics, Vision and Control v2 (RVC)
- notebook : chap3.ipynb
- Documentación Robotics Toolbox
- Git hub RVC v2